Improving 3D alpha wrapping by preserving concave features and reducing mesh complexity

Michel Beeren[a,1], Clara García-Sánchez[a], Hugo Ledoux[a]

[a] *3D Geoinformation Research Group, Delft University of Technology, Delft, The Netherlands*

Abstract

CGAL's 3D Alpha Wrap robustly produces valid enclosing meshes for defective input using two parameters: α, which controls the carving-ball size, and δ, an offset that controls the distance between the input and the output surface vertices. However, preserving sharp concave features often requires a small global α-value, which also increases refinement in regions where additional vertices do not substantially improve the approximation. This paper proposes a τ-guided extension that adds a local face-to-input distance criterion to the original algorithm. The new parameter τ triggers additional refinement only where the current wrap remains locally too far from the input.

To make this refinement effective near concave features, we introduce R3 as an additional vertex-insertion strategy alongside the two insertion strategies of the original algorithm. R3 aims to place new vertices more directly near sharp concave features, whereas the original two insertion strategies may reach comparable vertex positions only after repeated refinement. The extension preserves the carving-based structure of the original algorithm, and we prove that the guarantees of termination, strict enclosure, watertightness, and manifoldness still hold. Experiments on 3D BAG building data show that the proposed method improves both average and local approximation quality, achieving lower directed Chamfer distances with fewer output faces than the original algorithm. Although R3 is less time efficient per insertion, it places vertices more effectively near concave features, so fewer insertions are needed to reach lower approximation errors. As a result, the proposed method can reduce runtime in the tested use case.

Keywords: 3D Alpha Wrap; mesh repair; mesh approximation; concave feature preservation; local refinement; geometric validity; computational geometry

1. Introduction

Surface meshes are essential in many geometry-processing, reconstruction, and simulation workflows (Portaneri et al., 2022). For such applications, the input mesh should not only approximate the intended geometry accurately, but also be valid: watertight, orientable, and manifold. In practice, however, existing models often contain defects or unnecessary complexity. Manually repairing such meshes can be time-consuming, while automatically repairing arbitrary combinations of defects remains challenging (Attene et al., 2013; Pađen et al., 2024; Saeedrashed & Benim, 2019).

CGAL's 3D Alpha Wrap algorithm provides a robust global repair and approximation method for such inputs (Portaneri et al., 2022). The method wraps the input by carving a 3D Delaunay triangulation around an offset surface of the input. This carving process is controlled by the parameter α, which defines the radius of the α-ball: smaller α-values allow the wrap to pass through smaller cavities and openings. When carving is not possible without exposing the input, the triangulation is refined by inserting additional vertices on the offset surface. The resulting mesh is guaranteed to terminate, strictly enclose the input, and be watertight, orientable, and manifold. This makes the method

[1]Corresponding author.
E-mail addresses: m.beeren@student.tudelft.nl (M. Beeren), c.garcia-sanchez@tudelft.nl (C.García-Sánchez), h.ledoux@tudelft.nl (H. Ledoux)

This work is licensed under a Creative Commons Attribution 4.0 International License. To view a copy of this license, visit http://creativecommons.org/licenses/by/4.0/.

A thesis submitted to the Delft University of Technology in partial fulfillment of the requirements for the degree of Master of Science in Geomatics.

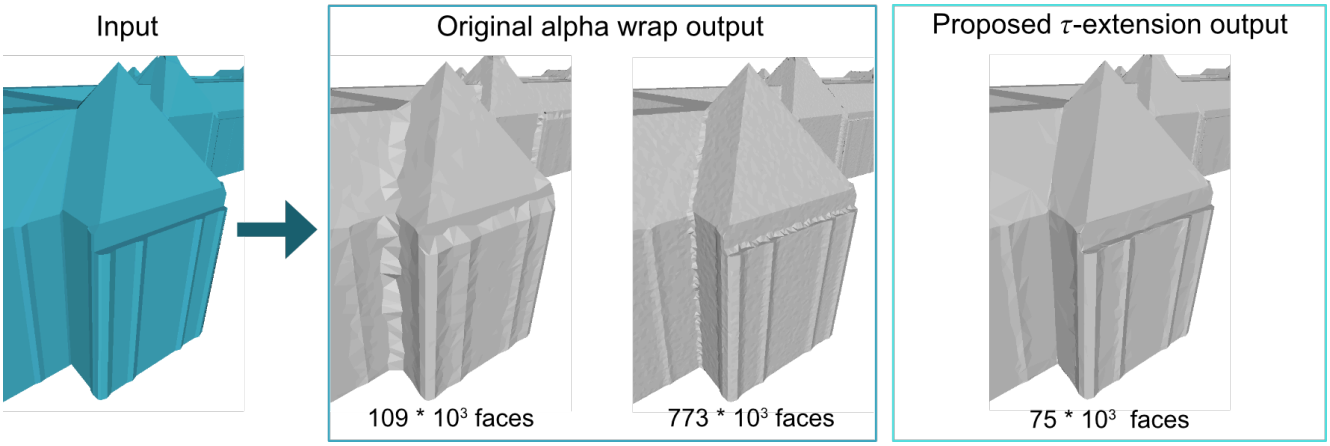


Figure 1.1: Comparison between the original CGAL 3D Alpha Wrap algorithm and the proposed τ-extended method for the Delft University of Technology Bouwkunde building from 3D BAG. The input is invalid and contains self-intersections and duplicated vertices, whereas the output meshes are watertight, manifold, and free of self-intersections. In the original method, sharp concave features are rounded; decreasing α makes them more defined, but does not fully remove this effect and increases mesh complexity globally. The proposed method approximates the sharp concave features more accurately with fewer output faces.

especially suitable for defective, overly complex, or otherwise difficult input geometries.

However, two limitations remain (see also Figure 1.1):

1. *Resolution/complexity trade-off*: Capturing small features typically requires a small α-value. This can improve the approximation of detailed regions. However, since α is a global parameter, the same small value also applies to smooth or nearly planar regions. As a result, the number of output faces increases throughout the mesh, including in regions where the additional faces do not substantially improve the approximation.
2. *Rounding of sharp concave features*: The α-ball cannot reach sharp concave regions as effectively as convex regions. Since the wrap is constructed from the outside, these concave features are therefore less well preserved and tend to appear rounded. A smaller α-value allows the algorithm to carve deeper into concave regions and improves their approximation, but sharp concave features may still remain rounded.

These limitations are especially relevant for inputs that combine large planar areas with sharp concave features, such as detailed building models used in flow simulation workflows, where geometric details and sharp edges can influence the simulation results (Pađen et al., 2024). For such inputs, decreasing α is not an ideal solution: it increases mesh complexity throughout the output, while sharp concave features may still remain rounded. This motivates a method that improves the approximation of sharp concave features without increasing mesh complexity globally.

This paper presents a τ-guided extension within the original 3D Alpha Wrap framework in CGAL. The main contributions are:

- A new precision parameter τ, which defines a maximum allowed facet-centroid-to-input distance and triggers additional local refinement when this threshold is exceeded.
- A new vertex-insertion strategy, R3, used for facets refined by the τ-criterion, designed to place refinement vertices more effectively near sharp concave features.

Additionally, we also provide a proof that the proposed extension retains the guarantees of the original method: termination, strict enclosure, watertightness, and manifoldness.

Together, these extensions aim to improve the geometric fidelity in difficult regions without requiring a globally smaller α. In this way, the method reduces run time and unnecessary mesh complexity while preserving the robustness and guarantees of the original 3D Alpha Wrap algorithm.

The remainder of this paper is structured as follows. Section 2 discusses related work on mesh repair, mesh approximation, and feature-preserving methods. Section 3 reviews the original 3D Alpha Wrap algorithm and presents the proposed τ-guided extension, including the new τ-criterion, the R3 insertion strategy, the theoretical guarantees, and implementation details. Section 4 evaluates the method on 3D BAG data in terms of approximation quality, output complexity, runtime, and validity. Section 5 concludes with the main findings, limitations, and future work.

2. Related Work

Real-world meshes often contain errors such as holes, self-intersections, non-manifold elements, or inconsistent orientations (Pađen et al., 2024). This is problematic because many applications and simulations require valid mesh input. Such meshes can be repaired or approximated, but it remains difficult to produce an output that is simultaneously valid, close to the input, and not overly refined (Attene et al., 2013).

This related work focuses on methods relevant to our setting: repair and approximation methods for defective input geometry, and feature-preserving methods that aim to improve sharp-feature approximation without unnecessarily increasing mesh complexity.

2.1. Repairing invalid surface meshes

Many *local* repair methods detect and correct specific defects directly. This can preserve the original geometry well, since only local parts of the mesh are changed. However, local repair methods often address only a subset of possible validity errors and can struggle when multiple defect types occur simultaneously, particularly for complex or heavily defective inputs (Attene et al., 2013). For this reason, this section focuses on global and hybrid repair methods.

Global repair methods reconstruct or approximate the complete surface rather than repairing each defect individually, which can make them more robust for defective inputs. Hybrid methods combine such global reconstruction with local projection or optimization, but these approaches still involve trade-offs between validity, approximation quality, feature preservation, and output complexity (Attene et al., 2013; Portaneri et al., 2022; Wen et al., 2025).

Volumetric reconstruction. Volumetric and implicit reconstruction methods convert the input to a grid, voxel, octree, or scalar-field representation and then extract a new surface from it (Berger et al., 2017). Such methods can be robust to local defects and holes (Bischoff et al., 2005). However, the output is resolution-dependent: coarse representations may smooth or remove sharp features, while fine representations increase computational cost and mesh complexity.

ManifoldPlus. ManifoldPlus follows a related hybrid strategy: it first extracts a surface from an occupied/empty volumetric representation and then applies projection-based optimization to better approximate the input (Huang et al., 2020). However, degenerate faces can occur, as also reported by (Portaneri et al., 2022), making the method less robust when it is used as a mesh repair method to obtain a valid output mesh.

Heuristic shrink wrapping. For 3D city models, (Zhao et al., 2013) propose HSW, a heuristic shrink-wrapping method for automatically repairing invalid solid-based CityGML LoD2 building models (Zhao et al., 2013). The method repairs and generalizes building models by shrink-wrapping a bounding surface towards the input. Their experiments show a strong reduction in invalid solids, although some repaired outputs remain invalid. Since the method is developed specifically for solid-based LoD2 building models, it is less directly applicable to more general input types such as triangle soups, point sets, or mixed defective geometry.

3D alpha wrap. CGAL's 3D Alpha Wrap is a robust global repair and approximation method that produces a watertight, orientable surface mesh that strictly encloses the input, even when the input contains defects (Portaneri et al., 2022). These guarantees make the method especially suitable for difficult repair cases. The method refines and carves a 3D Delaunay triangulation on an offset surface of the input. The output is controlled by two parameters: offset, which controls the distance between inserted vertices and the input, and α, which controls the carving scale.
However, as shown in Figure 1.1 for inputs with sharp concave features, two limitations remain relevant. First, since α is a global parameter, improving detailed regions often increases mesh complexity throughout the output. Second, sharp concave features may still remain rounded, even for smaller α-values.

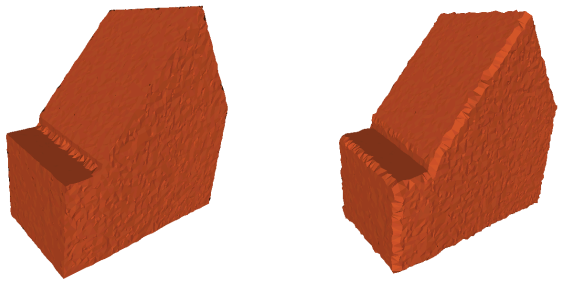
These limitations motivate the extension proposed in this work and are discussed in more detail in Section 3.

2.2. Feature-preserving methods

Besides validity, the repaired mesh should also preserve relevant geometric features. For building models, sharp edges and concave regions are important because rounding them changes the local shape of the object. Global repair methods may still smooth or round these features, even when they produce a valid output mesh. The following methods aim to improve sharp-feature preservation during or after repair.

Alpha wrapping from a cavity. CGAL's 3D Alpha Wrap also supports wrapping from an inner cavity (Portaneri et al., 2022). In the standard outer wrap, convex features are generally preserved better than concave features, because concave regions are approached from the outside and may be rounded. When wrapping from an inner cavity, this behavior is reversed: the same concave regions are approached from the inside and can therefore be preserved more sharply. However, combining the inner and outer wraps in a way that keeps the best-preserved sharp features from both surfaces, while still producing one valid output mesh,

is not straightforward. The two wraps may not align or intersect in suitable locations, merging them may cause intersections with the input, and the inner wrap may include internal structures that would need to be detected and removed.



(a) Outer wrap (b) Inner-cavity wrap

Figure 2.1: Comparison of outer and inner-cavity wrapping with CGAL's 3D Alpha Wrap algorithm.

Point optimization. (Wen et al., 2025) propose a feature-preserving repair method that first applies CGAL's 3D Alpha Wrap and then relocates sampled output vertices based on the underlying input geometry. The relocation step minimizes angular errors between neighboring vertices and tangent planes, which tends to pull vertices toward creases and corners. Although, this does improve the alignment of sharp features, especially in concave regions, it does not preserve the validity guarantees of the original Alpha Wrap. In particular, relocating vertices back toward the input may introduce self-intersections or invalid geometry, especially for thin or near-zero-volume structures. The method can also be computationally expensive, since the input must first be wrapped with a sufficiently small α, followed by an additional optimization step, whose computation time strongly depeds on the amount of sampled vertices used for this optimization step. Moreover, the method relies on input normal information, making it less suitable for inputs such as raw point clouds or triangle soups.

Protection balls. Some Delaunay-based meshing methods use protection balls around sharp features to prevent unwanted triangulation across edges and corners (Cheng et al., 2007). This can help preserve sharp features during refinement. However, these methods often rely on weighted Delaunay triangulations, which do not satisfy the same empty-sphere property as the ordinary Delaunay triangulation used in 3D Alpha Wrap. Therefore, directly integrating protection balls into the Alpha Wrap framework is not straightforward, since the termination guarantees of the original algorithm may no longer apply. In addition, the setting of Alpha Wrap is more difficult because refinement takes place on an offset surface. Sharp features would therefore not only need to be detected, but also projected to suitable offset locations, which is ambiguous for complex or defective input geometry (Portaneri et al., 2022).

3. Methodology

3.1. Original 3D Alpha Wrap algorithm

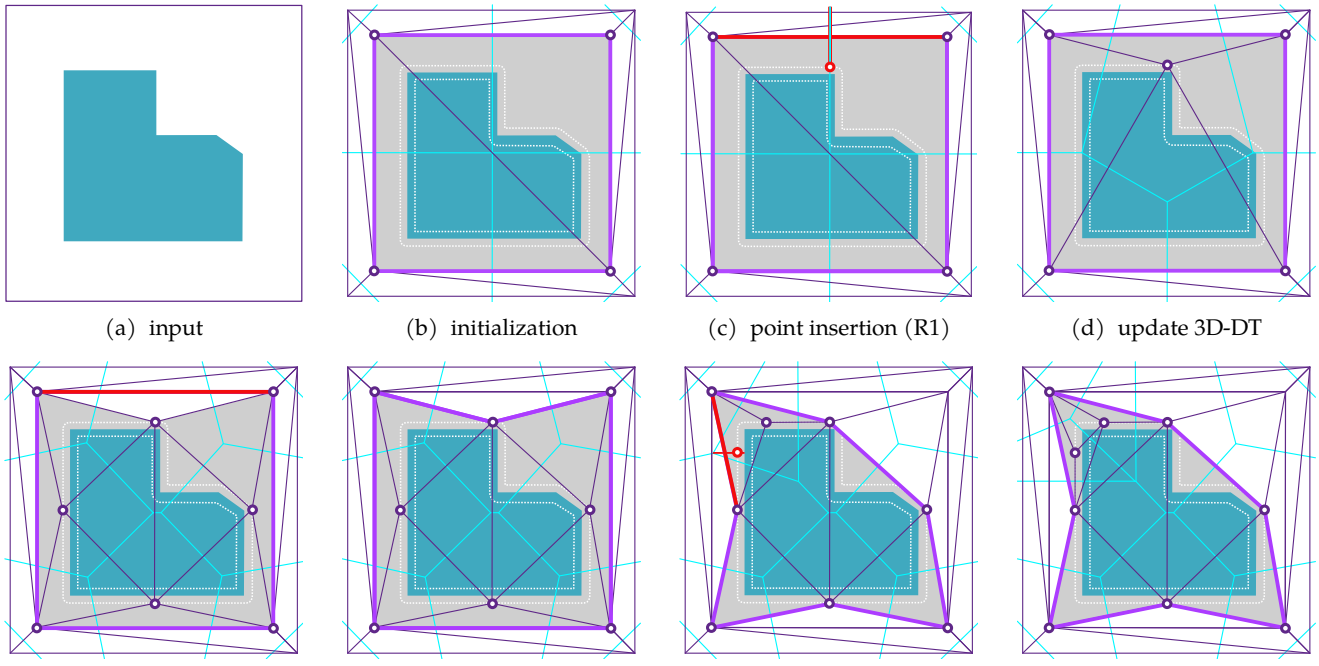
To clarify where the proposed extension adds to the original method, we first summarize the relevant concepts of CGAL's 3D Alpha Wrap algorithm.

Given a 3D input geometry, the original 3D Alpha Wrap algorithm is a robust global mesh repair and approximation algorithm. It is guaranteed to terminate and to generate a surface mesh that is manifold, watertight, and strictly encloses the input. These guarantees also hold for inputs that are unoriented, defective, or represented only as a triangle soup, point set or mixed input. The algorithm can be interpreted as a shrink-wrapping procedure around an offset surface, where a carving ball is used to shrink the mesh towards the input.

Parameters. The original algorithm is controlled by two global parameters: α and δ. The parameter α defines the radius of the carving ball: smaller α-values allow the wrap to pass through smaller cavities and openings. The parameter δ defines the distance of the implicit offset surface around the input. New vertices inserted during refinement are placed on this offset surface.

Initialization. A loose axis-aligned bounding box is first constructed around the input and its offset surface. This bounding box is tetrahedralized using a 3D Delaunay triangulation. The tetrahedra of this triangulation are referred to as cells. The cells that make up the bounding box are initially labelled INSIDE, while the infinite cells outside the bounding box are labelled OUTSIDE. The surface mesh is defined by the triangular facets separating an INSIDE cell from an OUTSIDE cell. At initialization, this surface is therefore the boundary of the loose bounding box, which strictly encloses the input (see Figure 3.1b).

Alpha wrapping. After initialization, the flood-filling procedure starts. The algorithm maintains a queue of facets that still need to be processed. Each facet in this queue is tested for α-traversability. A facet is called α -traversable when its minimal empty Delaunay ball is larger than the α-ball. Intuitively, this means that the facet is still large enough, relative to α, to be considered for further processing.



(a) input (b) initialization (c) point insertion (R1) (d) update 3D-DT

(e) carve through f (f) update surface mesh (g) point insertion (R2) (h) update 3D-DT

Figure 3.1: Schematic 2D illustration of the main steps of the original 3D Alpha Wrap algorithm, included to clarify where the proposed extension differs later in Figure 3.2 and Figure 3.3. For conciseness, only the main operations are shown. The input is shown in turquoise, the offset surface as a white dotted line, the current surface mesh in purple, the currently processed facet and inserted point in red, the Delaunay triangulation as thin dark-purple lines, and the Voronoi diagram in light blue. (b) A loose bounding box is initialized around the input. (c) For an α-traversable facet f, Rule R1 inserts a Steiner point when the dual Voronoi edge intersects the offset surface. (d) The Delaunay triangulation is updated after insertion. (e) If no such intersection exists and the adjacent INSIDE cell does not intersect the input, the cell is carved. (f) The current surface mesh is updated after carving. (g) If the dual Voronoi edge does not intersect the offset surface but carving would expose the input, Rule R2 inserts a Steiner point instead. (h) The Delaunay triangulation is updated after insertion.

For each α-traversable facet, the algorithm checks whether the adjacent INSIDE cell can be carved. Carving means relabelling this cell as OUTSIDE (see Figure 3.1e), thereby moving the current surface mesh inward (see Figure 3.1f). This is only allowed if the cell can be removed without crossing the input geometry. If carving is allowed, the surface mesh is updated and newly exposed surface facets are added to the queue. If not, the triangulation is refined by inserting a new vertex on the offset surface, called a Steiner point.

The original algorithm uses two insertion strategies to determine where a Steiner point is placed: Rule R1 and Rule R2. Rule R1 tries to insert a Steiner point at the intersection between the dual Voronoi edge of the processed facet and the offset surface (see Figure 3.1c). This generally leads to well-shaped facets. If R1 fails, Rule R2 first projects the circumcenter of the adjacent INSIDE cell onto the input. From this projected input point, a new point is then constructed on the offset surface (see Figure 3.1g). This strategy often performs well near convex features.

The algorithm continues alternating between carving and refinement until the queue is empty. At this point, no remaining surface facet is α-traversable. The final output mesh is formed exactly by those triangular facets that separate one INSIDE cell from one OUTSIDE cell

3.2. Proposed τ-extension for the 3D Alpha Wrap algorithm

We propose an extension of CGAL's 3D Alpha Wrap algorithm that preserves the original carving-based structure. The extension adds two components to the original algorithm: an additional τ-traversability criterion and a new vertex-insertion strategy, denoted as Rule R3. The τ-criterion is used to continue refinement when the current surface is still locally too far from the input, while R3 is designed to place Steiner points more effectively near sharp concave features. The main guarantees of the original method, namely termination, strict enclosure, watertightness, and manifoldness, are retained and discussed in Section 3.3.

τ-traversability. In the original algorithm, a surface facet is processed only when it is α-traversable. Once no remaining surface facet satisfies this criterion, the algorithm terminates. However, near sharp concave features, the current surface can still be relatively far from the input even when the α-criterion no longer triggers refinement.

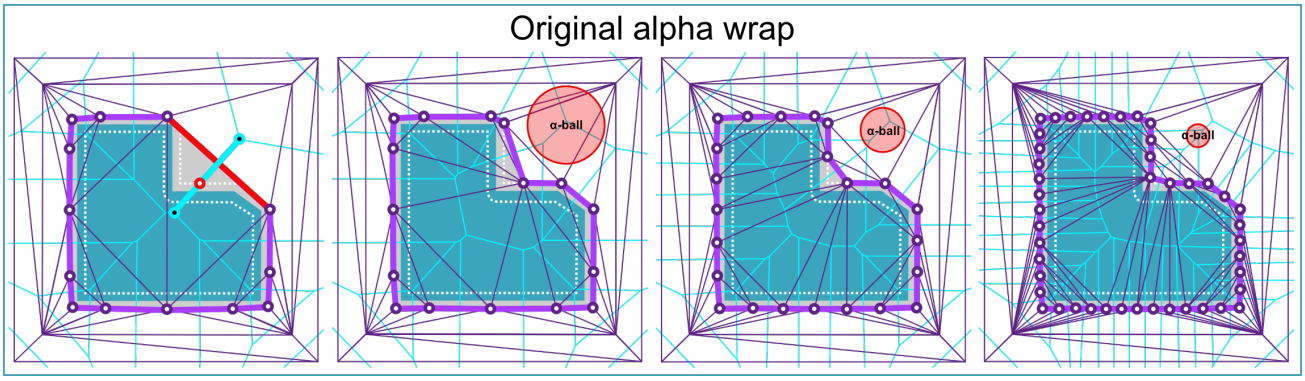


Figure 3.2: 2D illustration of output meshes generated by the original 3D Alpha Wrap algorithm using only the α-traversability criterion and insertion rules R1 and R2. The examples show decreasing α-ball sizes: smaller α-values improve the approximation of the concave feature, but also increase refinement in other regions. A zoomed-in view of the Steiner point insertion near the concave region is shown in Figure 3.6a.

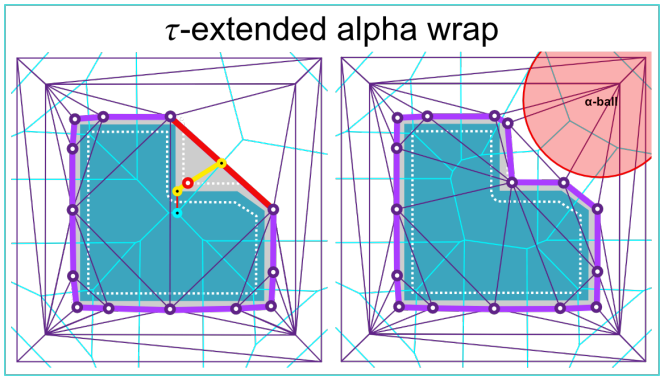
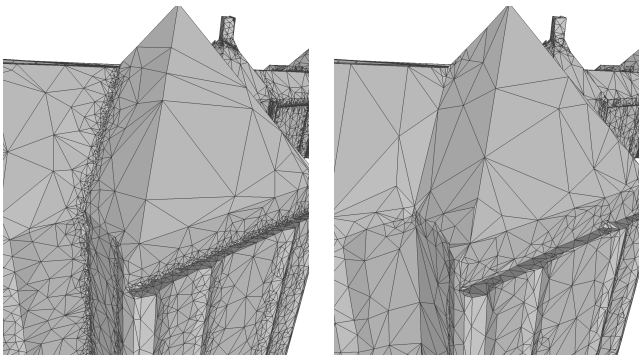


Figure 3.3: 2D illustration of output meshes generated by the proposed τ-extended 3D Alpha Wrap algorithm. The method adds the τ-traversability criterion and insertion rule R3, allowing additional refinement where the wrapped surface remains too far from the input. R3 places Steiner points more directly near the concave feature, so that the feature can be approximated more closely even with a larger α-ball. A zoomed-in view of the R3 insertion near the concave region is shown in Figure 3.6b.

We therefore introduce an additional traversability criterion controlled by a new global parameter τ. This parameter defines a maximum allowed face-to-input distance. For a facet f, with centroid g_f, we compute the shortest distance to the input geometry I, denoted as $d(g_f, I)$. If this distance exceeds the threshold τ, the facet is considered τ-traversable: Since inserted vertices lie on the offset surface, τ is chosen larger than the offset distance δ, so that the threshold is larger than the Steiner-point-to-input distance.

In our extended algorithm, a facet can be processed in two cases. If the facet is α-traversable, the original Alpha Wrap procedure is applied. If the facet is no longer α-traversable but still exceeds the τ-threshold, and thus is τ-traversable, the original algorithm would stop at this location, whereas the extension continues refinement locally. This allows regions that remain too far from the input to be refined without decreasing α globally (see Figure 3.2 vs Figure 3.3).

Rule R3. Adding the τ-traversability criterion alone is not sufficient, because the original insertion rules R1 and R2 are not specifically designed to place Steiner points near sharp concave features. In such regions, using only R1 and R2 may require many refinement steps before the surface becomes close enough to the input to satisfy the τ-threshold. We therefore introduce a third insertion rule, R3, which is used only for facets that are τ-traversable but not α-traversable (see Figure 3.4a vs Figure 3.4b).



(a) using only R1 and R2: $94.4 * 10^3$ Steiner points.

(b) using R1, R2 and R3: $36.3 * 10^3$ Steiner points.

Figure 3.4: Effect of insertion rule R3 near concave features, shown for the Delft University of Technology Bouwkunde building using $\alpha = 10$, offset $= 0.03$, and $\tau = 0.06$.

The goal of R3 is to place a Steiner point on the outer offset surface close to a sharp concave feature. The un-

derlying idea of the construction is to first estimate the location of the relevant concave feature on the input, and then determine a direction from this point towards the outer offset surface, so that a Steiner point can be placed where this direction reaches the offset surface.

Let f be a surface facet that is τ-traversable but not α -traversable, and let c_n be the adjacent INSIDE cell. Let k_{c_n} be the circumcenter of this cell.

First, R3 tries to target the relevant concave feature on the input. To do this, the closest point on the input to k_{c_n} is computed and denoted by P_p. Although P_p is not guaranteed to lie exactly on a sharp edge, it is expected to lie on or near the relevant concave feature.

The next step is to approximate a direction from this input region towards the outer offset surface.
In an idealized setting, a sharp concave feature can be seen as the intersection of neighboring input planes. A suitable point on the outer offset surface near this feature would then lie in a direction that is approximately equidistant from the neighboring planes, corresponding to the local bisector direction of the concave feature. If reliable normals and connectivity were available, this direction could be estimated directly from the incident input faces (see Figure 3.5). However, the input may be unoriented, defective, or represented only as a triangle soup or point set. Therefore, R3 approximates this direction using only closest-point queries to the input and the currently processed surface facet.

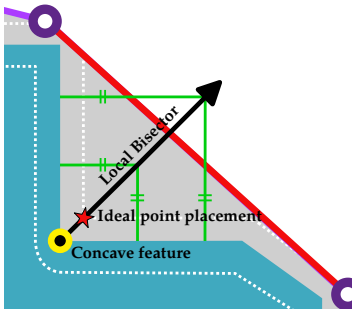
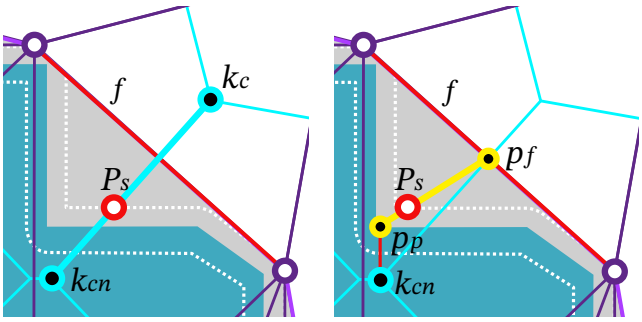


Figure 3.5: Idealized point placement near a sharp concave feature. The desired point on the outer offset surface lies approximately along the local bisector direction of the neighboring input planes.

As shown in Figure 3.6, to approximate this direction, R3 first selects a helper point on the currently processed facet f. The edge midpoints of f are evaluated, and the midpoint with the largest distance to the input is selected. This midpoint is used as an approximation of the point on f that lies farthest from the input. Together with the approximated concave feature location P_p, it defines a direction that serves as an approximation of the local bisector direction towards the outer offset surface.

Before constructing the segment, the selected helper point is shifted towards the facet centroid g_f by 5% of the vector from the helper point to g_f, yielding P_f. We require $d(P_f, I) > \delta$, so that P_f lies outside the offset region. The candidate Steiner point is then constructed using the segment from P_f to P_p. Starting from P_f, the first intersection of this segment with the offset surface is selected. This intersection is expected to give a good approximation of the desired Steiner point position on the outer offset surface near the concave feature (see Figure 3.6a vs Figure 3.6b).



(a) Insertion using R1; zoomed-in view of Figure 3.2.

(b) Insertion using R3; zoomed-in view of Figure 3.3.

Figure 3.6: Zoomed-in schematic comparison of the original R1 insertion rule and the proposed R3 insertion rule near a concave feature. R1 inserts a Steiner point at the intersection of the dual Voronoi edge with the offset surface, whereas R3 targets the concave region more directly using the segment from P_f to P_p.

Before insertion, the candidate point is tested against a lower bound on the insertion radius. The insertion radius is the shortest distance from the candidate Steiner point to the set of previously inserted Steiner points. As discussed in Section 3.3, the insertion radius must be bounded from below to prove termination. For R3, this lower bound is enforced explicitly: the candidate point is accepted only if its insertion radius is larger than the lower bound used for Steiner points inserted by R1 and R2, $\min(\alpha, \delta, \tau - \delta)$.

If this condition is not satisfied, R3 is rejected and the algorithm falls back to the original insertion rules R1 and R2. The 5% shift is applied to the construction point used to determine the R3 candidate. Moving the construction point towards g_f generally places it closer to the circumcenter of the neighboring cell. Because the Delaunay ball of that cell contains no previously inserted vertices, moving closer to its circumcenter generally increases the construction point's minimum distance to previously inserted Steiner points. Consequently, although the shift is not required for the algorithm to function, it makes R3 more likely to produce a candidate whose insertion radius meets the lower-bound requirement defined above, reducing the need to fall back to R1 or R2.

Algorithm 1: τ-Guided 3D Alpha Wrapping

Input: Input geometry I, Parameter α, Offset surface I_δ,
Parameter τ
Output: Output surface mesh $\mathcal{O}$
Initialize Delaunay triangulation $\mathcal{D}$ with the vertices of a loose bounding box of I_δ
Tag OUTSIDE all infinite cells of $\mathcal{D}$
Tag INSIDE all finite cells of $\mathcal{D}$
Initialize priority queue $\mathcal{Q}$ with convex hull facets of $\mathcal{D}$ that are α-traversable. $\mathcal{Q}$ is sorted in decreasing order of min Delaunay ball radius $\Phi(f)$
while $\mathcal{Q}$ *is not empty* **do**
  Gate $f \leftarrow \mathcal{Q}.top()$
  $\mathcal{Q}.pop()$
  if $\Phi(f) < \alpha$ **then** // *α-traversability check*
    Let f_{centroid} be the centroid of facet f
    Let d_I be the shortest distance from f_{centroid} to I
    if $d_I \leqq \tau$ **then** // *τ-traversability check*
      continue
  Let c_o be the cell incident to f tagged OUTSIDE
  Let c_n be the other cell incident to f
  if c_n *is tagged* OUTSIDE **then**
    continue
  Let k_c be the dual Voronoi vertex of c_o (circumsphere center), or the circumcircle of f if c_o is infinite
  Let k_{c_n} be the dual Voronoi vertex of c_n
  Let V_f be the oriented segment $[k_c\,;k_{c_n}]$
  Let P_s be a potential Steiner point
  if $\Phi(f) < \alpha$ **and** c_n *intersects* I **then**
    Let P_p be the point closest from k_{c_n} on I
    Let P_f be the edge midpoint of f farthest from I, shifted 5% towards its centroid.
    Let E_e be the oriented segment $[P_f\,;P_p]$
    Let $\mathcal{P}$ be the set of all previously inserted points
    Let P_{test} be a candidate Steiner point whose distance to $\mathcal{P}$ is checked before insertion
    if E_e *intersects* I_δ **then**
      $P_{\text{test}} \leftarrow$ first intersection point between E_e and I_δ
      Let d_p be the shortest distance from P_{test} to $\mathcal{P}$
      Let $d_{\min}$ be $\min(\alpha, \delta, \tau - \delta)$
      if $d_p > d_{\min}$ **then**
        $P_s \leftarrow P_{\text{test}}$ // *R3*
  if V_f *intersects* I_δ **and** P_s *not yet assigned* **then**
    $P_s \leftarrow$ first intersection point between V_f and I_δ // *R1*
  else if c_n *intersects* I **then**
    Let P_p be the point closest from k_{c_n} on I
    Let E_p be the oriented segment $[k_{c_n}\,;P_p]$
    $P_s \leftarrow$ closest point from k_{c_n} on I_δ and along E_p // *R2*
  end
  if P_s *exists* **then**
    Remove from $\mathcal{Q}$ the gates that will be destroyed upon insertion of P_s into $\mathcal{D}$
    Insert P_s into $\mathcal{D}$
    Tag new finite cells INSIDE and new infinite cells OUTSIDE
    Push new α-traversable gates to $\mathcal{Q}$
  else
    Tag c_n as OUTSIDE
    Push all α-traversable gates incident to c_n to $\mathcal{Q}$
  end
end
$\mathcal{O} \leftarrow$ surface triangle mesh represented facets separating INSIDE from OUTISDE cells
return $\mathcal{O}$

Algorithm 1: Pseudocode of the τ-extended algorithm. The original algorithm is shown in black, while the extensions introduced in this work are highlighted in green and marked by a green bar.

Taking the first intersection with the offset surface when moving from P_f towards P_p is important to avoid inserting Steiner points on inner structures. The point P_f lies on a surface facet, and is therefore incident to an OUTSIDE cell; it is consequently reachable from the exterior and not located on an inner structure. Since P_f is also chosen outside the offset region ($d(P_f, I) > \delta$), the first intersection places the new Steiner point on the outer boundary of the offset region, before the segment can reach the input or possible inner structures. This is necessary because, in the original algorithm, R1 is tested first to prevent Steiner points from being inserted on inner structures. Since the extended algorithm may evaluate R3 before R1, R3 must provide the same protection.

3.3. Guarantees

The original 3D Alpha Wrap algorithm provides several theoretical guarantees: the algorithm terminates, and the output mesh is watertight, manifold, and strictly encloses the input (Portaneri et al., 2022). The modifications proposed in this work do not alter the parts of the algorithm on which the watertightness, manifoldness, and enclosure arguments rely. Hence, these guarantees are preserved. Only the proof of termination must be adapted, since both the traversability criterion and the Steiner-point insertion strategy have been modified.

Termination. In the original paper, termination follows from two key facts. First, a Steiner point inserted for refinement lies inside the Delaunay ball of the cell that is intended to be refined, so that the refinement step is effective and breaks that cell. Second, the insertion radius, defined as the shortest distance from the candidate Steiner point to the set of previously inserted Steiner points, is bounded from below, meaning that every newly inserted Steiner point is at least this fixed minimum distance from the previously inserted vertices. Together, these two facts give termination through the same packing argument as in the original proof.

These facts are proven in the original algorithm under the original α-traversability criterion and the original Steiner-point insertion rules R1 and R2. Since the modified algorithm adds the τ-based traversability criterion and introduces a new insertion strategy, the original termination proof does not directly apply anymore. Therefore, the same two facts need to be proven again for the modified setting. An adapted proof is given in Appendix Section A.

Watertightness. The output mesh is still defined as the set of facets separating one cell labelled INSIDE from one cell labelled OUTSIDE. Since this definition and the

cell-labelling procedure are unchanged, the output remains watertight.

Manifoldness. Each carving operation still removes an entire tetrahedron. Hence, the original manifoldness argument remains valid.

Enclosure. The modified algorithm preserves the two ingredients needed for strict enclosure: it starts from a bounding box that contains the input, and cells are carved only when they do not intersect the input. Since these parts are unchanged, the output mesh still strictly encloses the input.

3.4. Implementation

The extension is implemented in C++ directly within the existing CGAL (The CGAL Project, 2021) 3D Alpha Wrap codebase. The only algorithmic additions are the τ-traversability criterion and the corresponding point-insertion strategy, denoted as Rule R3. These additions are integrated into the existing refinement framework rather than implemented as a separate geometry-processing pipeline.

For all operations that depend on the input geometry or the offset surface, the extension reuses query types already present in the original Alpha Wrap implementation. These include tetrahedron-input intersection tests, segment-offset-surface intersections, and projections of query points onto the input. Therefore, the extension does not introduce new types of input- or offset-surface queries.

This is important because the original Alpha Wrap implementation explicitly addresses practical correctness under floating-point arithmetic, filtered predicates, and approximate constructions. By reusing the same types of geometric operations, the extension remains within the numerical framework already considered by the original implementation.

4. Experiments

All experiments are performed on a laptop running macOS 26.3.1, with an Apple M4 chip, 24 GB of memory, and internal solid-state storage. The code is compiled using Apple Clang 17.0.0.

A direct parameter-wise comparison with the original Alpha Wrap algorithm is difficult, because the proposed extension introduces the additional parameter τ. Identical α and offset values do not necessarily result in comparable behavior. Therefore, the methods are compared mainly in terms of approximation quality, output complexity, runtime, and validity. Output complexity is measured as the total number of output faces. Approximation quality is evaluated using the directed Chamfer distance and cumulative distance distributions.

All approximation-quality and runtime experiments are performed on the Delft University of Technology Aula building from the 3D BAG dataset (see Figure 4.1). This input is invalid according to the validity checks, making it a relevant real-world repair case. It also represents the intended use case of the extension, since it contains large planar regions combined with several sharp concave features. Such inputs are challenging for the original Alpha Wrap algorithm: accurately approximating concave regions requires a smaller global α, which also increases refinement in planar regions where this is not needed.

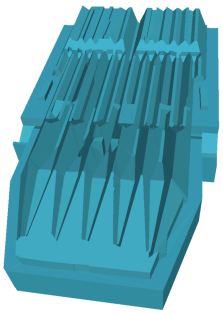


Figure 4.1: Delft University of Technology Aula building from the 3D BAG dataset, used for all approximation-quality and runtime experiments.

This targeted test case is chosen because it contains the type of geometry for which the extension is designed. The proposed method is not intended to change the behavior of Alpha Wrap in regions where the original wrap is already close enough to the input, meaning that the face-centroid-to-input distance does not exceed the τ-threshold. Therefore, for inputs where the original algorithm already remains within this threshold, or when τ is chosen sufficiently large, the extension follows the same refinement steps as the original algorithm and produces the same output mesh. In such cases, the only additional cost is evaluating the τ-criterion.

The validation experiments use additional datasets to test robustness more broadly. These include all 3D BAG buildings in Delft (Peters et al., 2022), increasingly corrupted input geometry, and a complete 3D BAG tile of the Delft University of Technology campus.

4.1. Approximation quality and output complexity

Since a fair comparison using identical parameters is not meaningful, the original and extended methods are compared based on the relation between approximation quality and output complexity. For the original Alpha Wrap algorithm, we vary the α-value. For the

τ-extended method, we use a large fixed α-value and vary τ, so that the influence of the new criterion can be studied more directly. The offset distance is kept fixed, since it strongly affects the distance between the output mesh and the input.

Approximation quality is measured using a directed Chamfer distance from the output mesh to the input, rather than a symmetric Chamfer distance. This avoids penalizing internal structures that may be present in the input but are intentionally not reconstructed in the output. For each measurement, 10,000 points are randomly sampled on the output mesh. For each sampled point, the shortest distance to the input mesh is computed. This process is repeated 100 times, after which the final reported value is obtained by averaging over all runs.

The effect of τ is first evaluated by plotting the directed Chamfer distance against the τ-value (see Figure 4.2). As τ increases, the result approaches the original Alpha Wrap output. This is expected: when the τ -threshold is sufficiently large, the additional refinement criterion is no longer triggered, and the method behaves like the original algorithm (see also Appendix Section E).

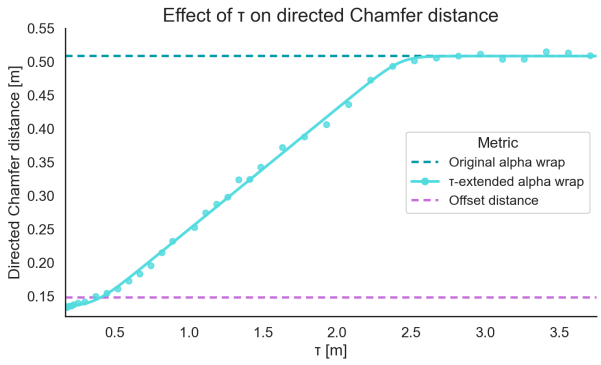


Figure 4.2: Directed Chamfer distance from the output mesh to the input for increasing τ-values. Both methods use $\alpha = 7.4$ and offset $= 0.15$. As τ increases, the additional refinement criterion is triggered less often and the result approaches the original Alpha Wrap output.

To study local approximation quality, we also use cumulative distance distributions (see Figure 4.3). The Chamfer distance provides a single average value, but does not show how errors are distributed over the mesh. The cumulative distribution shows that many sampled points lie close to the offset distance, which is expected since both methods insert points on the offset surface. However, lower τ-values reduce the number of poorly approximated regions. For example, for $\tau = 0.30$, more than 90% of sampled points lie within $0.2m$ of the input, whereas this is only about 60% for the original algorithm.

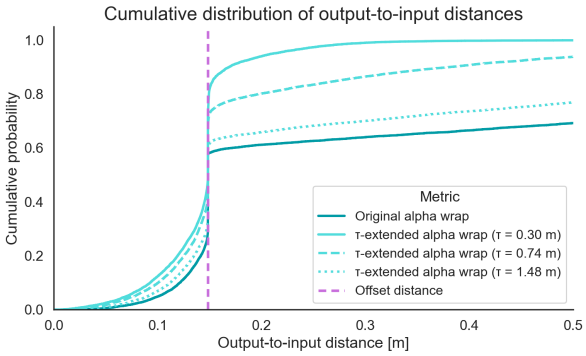


Figure 4.3: Cumulative distribution of sampled output-to-input distances. All runs use $\alpha = 7.4$ and offset $= 0.15$. Lower τ-values place a larger fraction of the output surface close to the input, indicating improved local approximation quality.

Finally, we compare directed Chamfer distance against output complexity (see Figure 4.4 and as visual results represented in Figure 4.5 and Figure 4.6). The original method is evaluated using a sweep over α, while the extended method is evaluated using a sweep over τ with a large fixed α. Smaller τ-values require more local refinement, but the new R3 insertion strategy aims to place refinement points more effectively near sharp concave regions. The results show that the τ-extended method reaches low directed Chamfer distances with fewer output faces than the original method.

As shown in Figure 4.4, the directed Chamfer distance can fall below the offset distance at higher output complexities. Although Steiner points are inserted at the prescribed offset, the connecting facets are not constrained to lie at this distance. The τ-criterion only triggers further refinement for facets that lie farther from the input than the prescribed offset. Consequently, facets that lie closer to the input remain unrefined and therefore lower the average directed Chamfer distance.

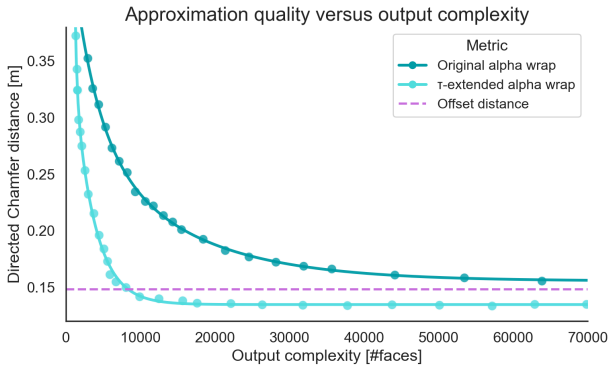


Figure 4.4: Directed Chamfer distance as a function of output complexity. The original Alpha Wrap method is evaluated by varying α, while the τ-extended method is evaluated by varying τ. The offset is fixed at 0.15, and the τ-extended method uses $\alpha = 7.4$. The τ-extended method reaches lower directed Chamfer distances with fewer output faces.

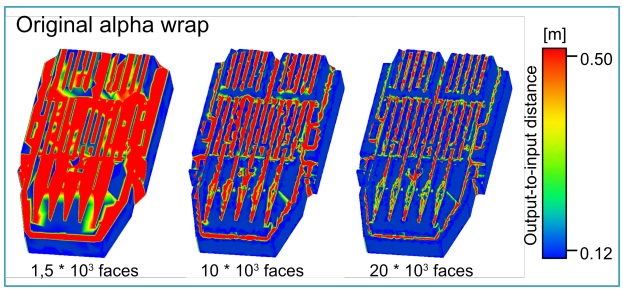


Figure 4.5: Visual results of the original Alpha Wrap algorithm for increasing output complexity. The three outputs contain approximately $1.5 * 10^3$, $10 * 10^3$, and $20 * 10^3$ faces.

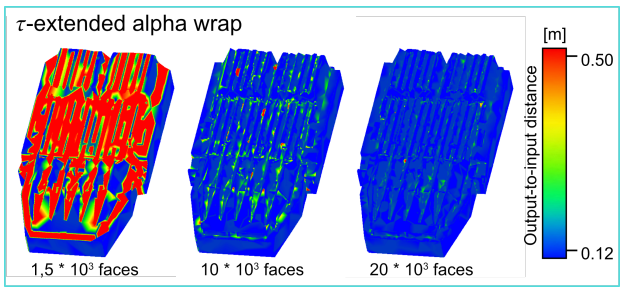


Figure 4.6: Visual results of the τ-extended Alpha Wrap algorithm for increasing output complexity. The three outputs contain approximately $1.5 * 10^3$, $10 * 10^3$, and $20 * 10^3$ faces.

4.2. Computational efficiency

Runtime is measured by repeating each wrapping experiment five times and taking the average.

We first compare output complexity against wrapping time (see Figure 4.7). In this experiment, larger α-values are used for the τ-extended method, so that refinement is more often triggered by the τ-criterion rather than by the original α-criterion. As a result, the new R3 insertion strategy is used more frequently. The plot shows that, in these cases, output complexity increases more slowly with wrapping time. This suggests that R3 is computationally more expensive than the original R1 and R2 insertion strategies: fewer output faces are generated per unit of time. This is expected, since R3 requires additional distance queries to the input.

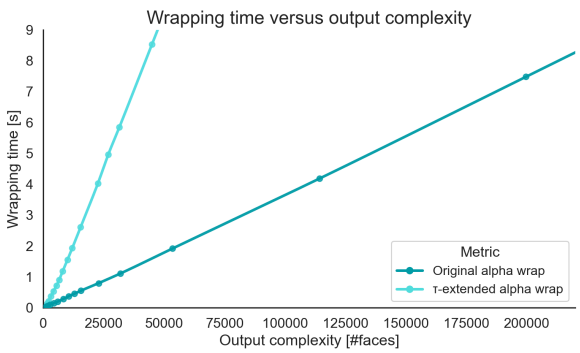


Figure 4.7: Wrapping time versus output complexity. The original Alpha Wrap varies α, while the τ-extended method varies τ with $\alpha = 7.4$; both use offset $= 0.15$. Its higher wrapping time at comparable output complexity suggests that R3 insertions are more expensive than R1 and R2.

We then compare the directed Chamfer distance against wrapping time. As suggested by Figure 4.7, R3 insertions are more expensive than the original R1 and R2 insertions. However, as shown in Figure 4.4, the τ-extended method requires fewer output faces to achieve a similar approximation quality. As a result, it reaches lower directed Chamfer distances faster than the original method in the tested use case (see Figure 4.8). This indicates that the additional cost of R3 can be compensated by more effective local refinement.

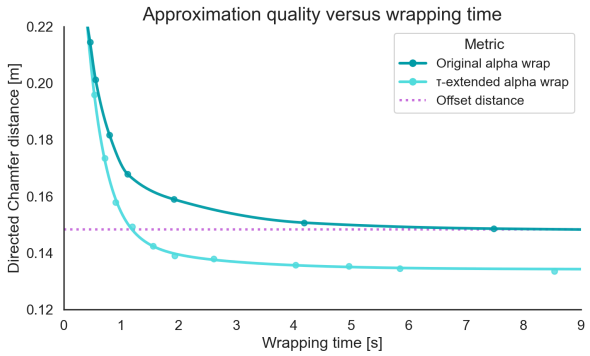


Figure 4.8: Directed Chamfer distance as a function of wrapping time. The original Alpha Wrap method is evaluated by varying α, with offset $= 0.15$. The τ-extended method uses $\alpha = 7.4$, offset $= 0.15$, and varying τ-values. Although R3 insertions are more expensive, the τ-extended method reaches lower directed Chamfer distances faster in the tested use case.

4.3. Validation

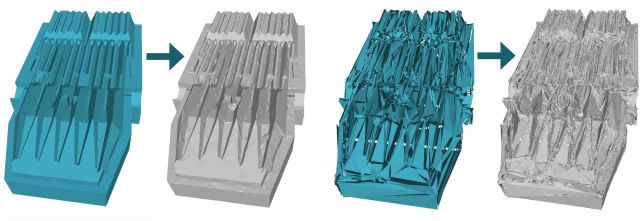
To evaluate whether the practical guarantees of the original Alpha Wrap algorithm are preserved, we test termination, watertightness and manifoldness. Termination is verified by whether the wrapping process finishes successfully. Watertightness and manifoldness are evaluated using val3dity (Ledoux, 2013; 2018).

Because some outputs may consist of multiple disconnected components, validity is evaluated per connected component. This avoids treating a valid collection of disconnected solids as one invalid solid. Each connected component is therefore tested individually with val3dity. In addition, possible intersections between disconnected components are checked using CGAL with exact predicates.

Three validation experiments are performed.

- **Test 1:** all 37,417 3D BAG buildings in Delft are wrapped individually using $\alpha = 10m$, offset $= 0.05m$, and $\tau = 0.1m$. This test evaluates whether the method consistently terminates and produces valid output across a large number of real-world building inputs, which may contain geometric defects. All buildings terminate successfully and pass the validity tests.

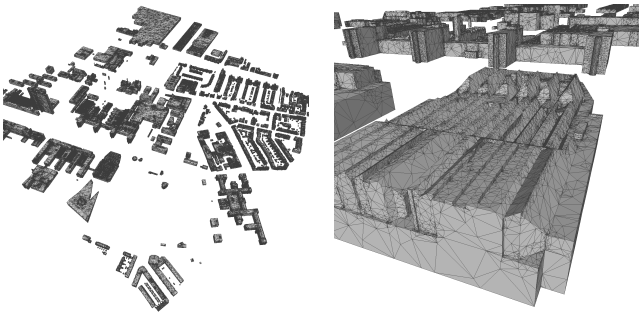
- **Test 2:** a robustness test is performed on increasingly corrupted input geometry. The input faces are first disconnected from each other and its vertices are then displaced in random directions with increasing displacement magnitudes. Even for a maximum displacement of 1m, the algorithm terminates and produces a valid output. However, at large displacements the input contains large openings, through which the wrap may enter the object and include internal structures. This is a limitation of both the original and extended methods. In the original Alpha Wrap algorithm, this behavior is mainly controlled by the α-value: a smaller α -ball can pass through smaller openings. In the τ-extended method, this effect additionally depends on the τ-threshold, since a smaller τ can trigger further refinement near openings and thereby allow the wrap to enter regions that would otherwise remain unwrapped.



(a) No random vector shift (b) Random vector shift of $1m$

Figure 4.9: Robustness test on increasingly corrupted input geometry. The input faces are first disconnected from each other, after which their vertices are displaced in random directions. Both examples are wrapped using $\alpha = 7.4$, offset $= 0.15$, and $\tau = 0.3$.

- **Test 3:** a complete 3D BAG tile of the Delft University of Technology campus is wrapped in a single run. The tile contains many separate buildings, including objects that are close to each other, touching, or partially overlapping. The algorithm must therefore process multiple disconnected components within the same wrapping procedure, rather than handling each object in isolation. The dataset also contains many sharp concave regions. A large α-value is combined with a small offset and small τ-value to trigger the new refinement criterion frequently. The algorithm terminates successfully and produces a valid output.



(a) Complete wrapped tile (b) Zoomed-in view

Figure 4.10: Wrapping result for the complete Delft University of Technology campus 3D BAG tile. The tile is wrapped as a single input using $\alpha = 10$, offset $= 0.025$, and $\tau = 0.05$.

5. Conclusions

This paper presents a τ-guided extension of CGAL's 3D Alpha Wrap algorithm-a simple and yet effective extension-designed to improve the preservation of sharp concave features without requiring a globally small α -value. The method introduces an additional τ-traversability criterion and a new vertex-insertion strategy, R3, while preserving the carving-based structure of the original algorithm.

A theoretical proof is provided showing that the main guarantees of the original 3D Alpha Wrap algorithm —termination, watertightness, manifoldness, and enclosure—are preserved. The arguments for watertightness, manifoldness, and enclosure carry over from the original method, since the relevant parts of the algorithm are unchanged. Only the termination proof is adapted to account for the new τ-traversability criterion and the R3 insertion rule.

The experiments on the Delft University of Technology Aula building show that the τ-extended method improves both the average and local approximation of the input. Compared to the original Alpha Wrap algorithm, it reaches lower directed Chamfer distances and, as shown by the cumulative distance distributions, has a larger fraction of sampled points close to the input, while using fewer output faces. This suggests that the τ-criterion and R3 insertion strategy refine the mesh more selectively: additional vertices are placed mainly in regions where the original wrap remains too far from the input, instead of increasing refinement globally through a smaller α.

The computational results show that the R3 insertion strategy is more expensive than the original R1 and R2 strategies. However, because the extension requires fewer output faces to reach a similar approximation quality, this additional cost can be compensated by more effective local refinement. As a result, the proposed method reaches lower directed Chamfer dis-

tances faster than the original method in the tested use case.

The validation experiments support the theoretical results in practice. The method terminates successfully and produces valid outputs for individual 3D BAG buildings in Delft, increasingly corrupted input geometry, and a complete 3D BAG tile of the Delft University of Technology campus.

Overall, the proposed extension is most useful for inputs that combine large planar regions with sharp concave features. For inputs where the original Alpha Wrap algorithm already performs well, or when τ is chosen sufficiently large, the τ-criterion is rarely triggered and the method behaves similarly to the original algorithm, apart from the additional cost of evaluating the criterion.

5.1. Limitations and future work

τ-criterion. The current τ-criterion uses the face-centroid-to-input distance as an approximation of the local surface-to-input distance. This is simple and computationally efficient, and works well in the tested cases. However, it can miss cases where the centroid lies close to the input, while another part of the same face is still relatively far away. A more robust, but still efficient, way of determining whether a local mesh face is sufficiently close to the input would therefore be a useful improvement.

R3 insertion strategy. The R3 insertion strategy improves vertex placement near sharp concave features, but is computationally expensive due to the additional distance-to-input queries and insertion-radius check. Although, the output is guaranteed to be watertight and manifold, R3 can produce less regular triangles or thin surface regions. Future work could further improve the vertex-placement strategy, while preserving the guarantees of the original Alpha Wrap algorithm. For example by improving the local bisector direction approximation.

Wrapping internal structures through input holes. As in the original Alpha Wrap algorithm, the wrap may enter the input through large holes or openings and include internal structures. In the original method, this behavior is mainly controlled by the α-value: smaller α-balls can pass through smaller openings. In the τ-extended method, it also depends on the τ-threshold, since smaller τ-values may trigger additional refinement near openings and increase the chance of wrapping internal structures.

Output simplification. The method may introduce more vertices than necessary near small or sharp concave features. This can happen when a vertex is inserted close to, but not exactly at, an optimal location, followed by another nearby vertex insertion in a later refinement step. A post-processing simplification step could remove such redundant vertices, provided that the approximation quality, enclosure, validity and termination guarantees are preserved.

Bibliography

Attene, M., Campen, M., & Kobbelt, L. (2013). Polygon mesh repairing: An application perspective. *ACM Computing Surveys (CSUR)*, *45*. https://doi.org/10.1145/2431211.2431214

Berger, M., Tagliasacchi, A., Seversky, L. M., Alliez, P., Guennebaud, G., Levine, J. A., Sharf, A., & Silva, C. T. (2017). A Survey of Surface Reconstruction from Point Clouds. *Computer Graphics Forum*, *36*(1), 301–329. https://doi.org/10.1111/cgf.12802

Bischoff, S., Pavic, D., & Kobbelt, L. (2005). Automatic Restoration of Polygon Models. *ACM Trans. Graph.*, *24*, 1332–1352. https://doi.org/10.1145/1095878.1095883

The CGAL Project. (2021). *CGAL User and Reference Manual* (5.3). https://doc.cgal.org/5.3/Manual/packages.html

Cheng, S.-W., Dey, T., & Levine, J. (2007). A Practical Delaunay Meshing Algorithm for a Large Class of Domains*. *Proceedings of the 16th International Meshing Roundtable*, 477–494. https://doi.org/10.1007/978-3-540-75103-8_27

Huang, J., Zhou, Y., & Guibas, L. (2020). ManifoldPlus: A Robust and Scalable Watertight Manifold Surface Generation Method for Triangle Soups. *Arxiv*. https://doi.org/10.48550/arXiv.2005.11621

Ledoux, H. (2013). On the Validation of Solids Represented with the International Standards for Geographic Information. *Computer-Aided Civil and Infrastructure Engineering*, *28*(9), 693–706. https://doi.org/10.1111/mice.12043

Ledoux, H. (2018). Val3dity: Validation of 3D GIS Primitives According to the International Standards. *Open Geospatial Data, Software and Standards*, *3*(1), 1. https://doi.org/10.1186/s40965-018-0043-x

Pađen, I., Peters, R., García-Sánchez, C., & Ledoux, H. (2024). Automatic High-Detailed Building Reconstruction Workflow for Urban Microscale Simulations. *Building and Environment*, *265*, 111978. https://doi.org/10.1016/j.buildenv.2024.111978

Peters, R., Dukai, B., Vitalis, S., van Liempt, J., & Stoter, J. (2022). Automated 3D reconstruction of LoD2

and LoD1 models for all 10 million buildings of the Netherlands. *Photogrammetric Engineering and Remote Sensing*, *88*(3), 165–170. https://doi.org/10.14358/PERS.21-00032R2

Portaneri, C., Rouxel-Labbé, M., Hemmer, M., Cohen-Steiner, D., & Alliez, P. (2022). Alpha Wrapping with an Offset. *ACM Trans. Graph.*, *41*(4), 127:1–127:22. https://doi.org/10.1145/3528223.3530152

Saeedrashed, Y. S., & Benim, A. C. (2019). Validation Methods of Geometric 3D-CityGML Data for Urban Wind Simulations. *E3s Web of Conferences*, *128*, 10006. https://doi.org/10.1051/e3sconf/201912810006

ShrinkWrap | Rhino 3-D Modeling. (n.d.). Retrieved January 17, 2026, from https://docs.mcneel.com/rhino/8/help/en-us/commands/shrinkwrap.htm

Wen, H., He, G., Xu, R., Chen, S., Xin, S., Shu, Z., Komura, T., Feng, J., Wang, W., & Tu, C. (2025). Feature-Preserving Mesh Repair via Restricted Power Diagram. *Proceedings of the Special Interest Group on Computer Graphics and Interactive Techniques Conference Conference Papers, SIGGRAPH Conference Papers '25*, 1–11. https://doi.org/10.1145/3721238.3730671

Zhao, Z., Ledoux, H., & Stoter, J. (2013). Automatic Repair of CityGML LOD2 Buildings using Shrink-Wrapping. *ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, *II-2/W1*, 309–317. https://doi.org/10.5194/isprsannals-II-2-W1-309-2013

A Proof of termination

We adapt the proof of Appendix B of the 3D Alpha Wrap paper ((Portaneri et al., 2022)) to the extended traversability criterion and to the additional insertion rule R3, while keeping the notation of Appendix B. In particular we denote c and c_n as the two cells incident to a traversable facet f, where c is a cell labelled OUTSIDE and c_n is a cell labelled INSIDE. We also use the same notation of insertion radius: for a point p inserted in the current vertex set $\mathcal{P}$, its insertion radius $r(p)$ is the Euclidean distance from p to $\mathcal{P}$ before insertion.

Let f be a finite facet and let $\Phi(f)$ denote the minimum radius of the Delaunay balls of f, as denoted in the original paper. In the extended algorithm, f is declared traversable if either $\Phi(f) \geq \alpha$ or $d(g_f, I) > \tau$, $\tau > \delta$, where g_f is the centroid of f and $\tau > \delta > 0$.

Furthermore let p_f be a point constructed in the interior of f, let k_{c_n} denote the circumcenter of the Delaunay ball of c_n, and let $\Pi_{I(k_{c_n})}$ be the closest point on I to k_{c_n}.

We introduce the following additional insertion rule.

- Rule R3. If c_n intersects I, $d(g_f, I) > \tau$, $\tau > \delta$ and $\Phi(f) < \alpha$, insert the intersection between the segment $[p_f; \Pi_{I(k_{c_n})}]$ and I_δ closest to p_f, if it exists and satisfies $r(p) \geq \min(\alpha, \delta, \tau - \delta)$. Otherwise, fall back to Rule R1 or Rule R2.

The next two lemmas are the extended counterparts of Lemmas 1 and 2 of Appendix B of the 3D Alpha Wrap paper.

Lemma 1: The insertion of a new point using Rule R3 breaks cell c_n.

PROOF. Let $\mathcal{B}_{c_n}$ be the Delaunay ball of c_n, centered at k_{c_n}. Since $c_n \subseteq \mathcal{B}_{c_n}$ and $c_n \cap I \neq \emptyset$, the closest point $\Pi_{I(k_{c_n})}$ belongs to $\mathcal{B}_{c_n}$. Moreover, p_f is constructed in the interior of f. Since f is a facet of c_n, its vertices lie on the boundary of $\mathcal{B}_{c_n}$, and every interior point of f lies in the interior of $\mathcal{B}_{c_n}$. Hence $p_f \in \text{int}(\mathcal{B}_{c_n})$. By convexity of $\mathcal{B}_{c_n}$, the segment $[p_f; \Pi_{I(k_{c_n})}]$ is contained in $\mathcal{B}_{c_n}$, and its open part is contained in $\text{int}(\mathcal{B}_{c_n})$. Therefore, any point inserted by rule R3 lies in $\text{int}(\mathcal{B}_{c_n})$, and thus breaks c_n. □

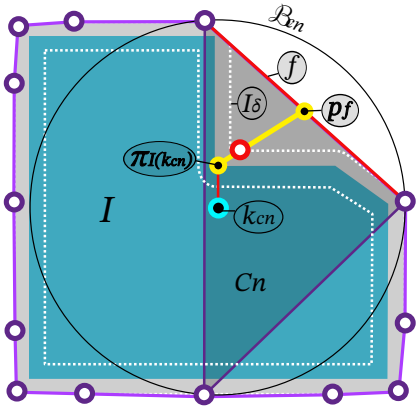


Figure A.1: Nomenclature for Lemma 1.

Lemma 2: The insertion radius of all inserted points in bounded from below: $\forall p \in \mathcal{P}$, $r(p) \geq \min(\alpha, \delta, \tau - \delta)$.

PROOF. Let p be an inserted point. If p is inserted by Rule R1, then, exactly as in the original proof, $r(p) \geq \Phi(f)$, where f is the traversable facet that triggers the insertion. If f is traversable because $\Phi(f) \geq \alpha$, then immediately $r(p) \geq \alpha$.

Assume now that f is traversable because $d(g_f, I) > \tau$. Facets incident to the initial loose bounding box, will not get checked for traversability. Hence, when a facet is declared traversable, we may restrict to facets whose vertices are not bounding-box vertices. All vertices inserted after initialization are created on I_δ. Therefore, if v_1, v_2, v_3 denote vertices of f, then $d(v_i, I) = \delta$, $i = 1, 2, 3$.

The distance function to I is 1-Lipschitz. Hence, for any two points x and y, $|d(x, I) - d(y, I)| \leq \| x - y \|$. Apply-

ing this with $x = g_f$ and $y = v_i$, we obtain $\| g_f - v_i \| \geq d(g_f, I) - d(v_i, I) > \tau - \delta$. Hence, the distance from g_f to each vertex of f is at least $\tau - \delta$, which yields $\frac{1}{3}\sum_{i=1}^{3} \| v_i - g_f \|^2 > (\tau - \delta)^2$.

Let R_f be the circumradius of f, and let o_f be its circumcenter. Since $\frac{1}{3}\sum_{i=1}^{3} \| v_i - g_f \|^2 = R_f^2 - \| o_f - g_f \|^2 \leq R_f^2$, it follows that $R_f > \tau - \delta$. Since $\Phi(f) \geq R_f$, it follows that, for this Rule R1 insertion triggered by a facet traversable under the τ-criterion, $r(p) \geq \Phi(f) > \tau - \delta$.

If p is inserted by Rule R2, the original proof applies unchanged, and $r(p) \geq \delta$.

If p is inserted by Rule R3, it is accepted only if $r(p) \geq \min(\alpha, \delta, \tau - \delta)$.

This proves the claim. □

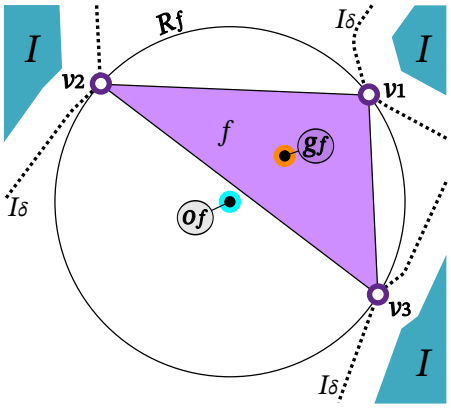


Figure A.2: Nomenclature for Lemma 2.

Theorem 1

The extended algorithm terminates.

PROOF. By Lemma 2, every inserted point has insertion radius at least $\rho = \min(\alpha, \delta, \tau - \delta) > 0$. The rest of the proof is identical to that of Appendix B of the 3D Alpha Wrap paper ((Portaneri et al., 2022)): a bounded region of $\mathbb{R}^3$ can contain only finitely many pairwise disjoint open balls of fixed radius $\frac{\rho}{2}$. Hence only finitely many insertions are possible, and the algorithm terminates. □

B Comparison Rhino ShrinkWrap

This appendix provides a qualitative comparison between the proposed τ-extended 3D Alpha Wrap method and Rhino's ShrinkWrap command. Rhino ShrinkWrap is included as a practical reference, since it can create closed meshes from defective or difficult-to-repair geometry (*ShrinkWrap | Rhino 3-D Modeling*, n.d.). Because the method is not open source, the comparison is intended as a visual reference rather than a direct algorithmic comparison.

The comparison is performed on the Delft University of Technology Aula building from 3D BAG. Rhino ShrinkWrap was run with a target edge length of $0.1m$ and polygon optimization set to 90%, resulting in $468,786$ faces. The τ-extended method was run with $\alpha = 7.4$, offset $= 0.03$, and $\tau = 0.06$, resulting in $37,420$ output faces.

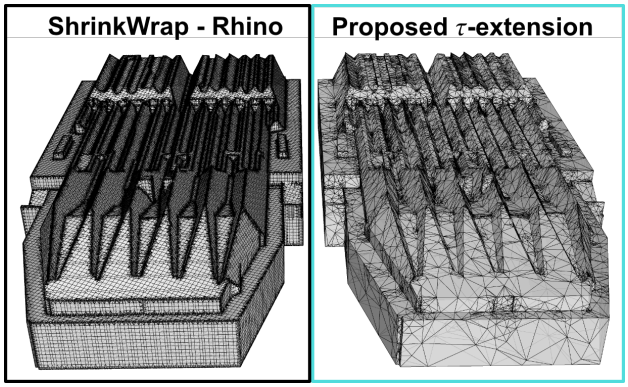


Figure B.1: Qualitative comparison between Rhino ShrinkWrap and the proposed τ-extended 3D Alpha Wrap method for the Delft University of Technology Bouwkunde building. Despite producing substantially more faces, the ShrinkWrap result shows more rounded sharp features.

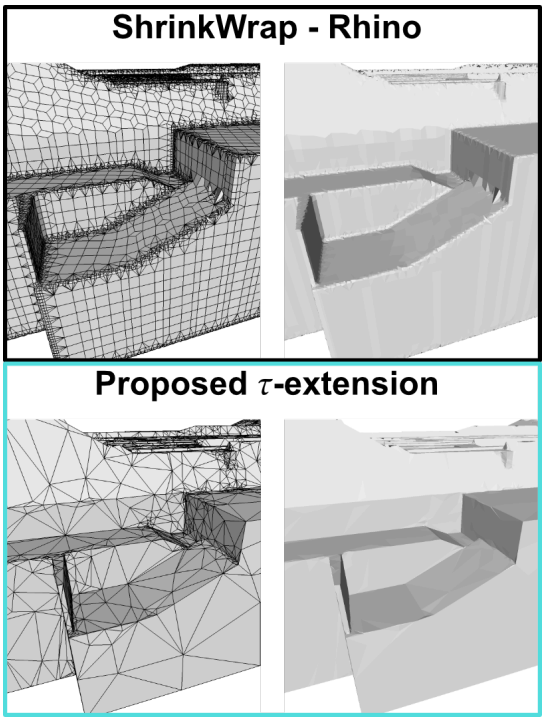


Figure B.2: Zoomed-in comparison shown with and without mesh edges. The edge views show the higher mesh density of Rhino ShrinkWrap, while the views without edges show that both convex and concave sharp features remain more rounded than in the τ-extended result.

Rhino ShrinkWrap can produce a closed output mesh, but both convex and concave sharp features remain rounded even with a small target edge length and polygon optimization. The τ-extended method preserves these features more clearly while using fewer output faces.

E Qualitative effect of α and τ

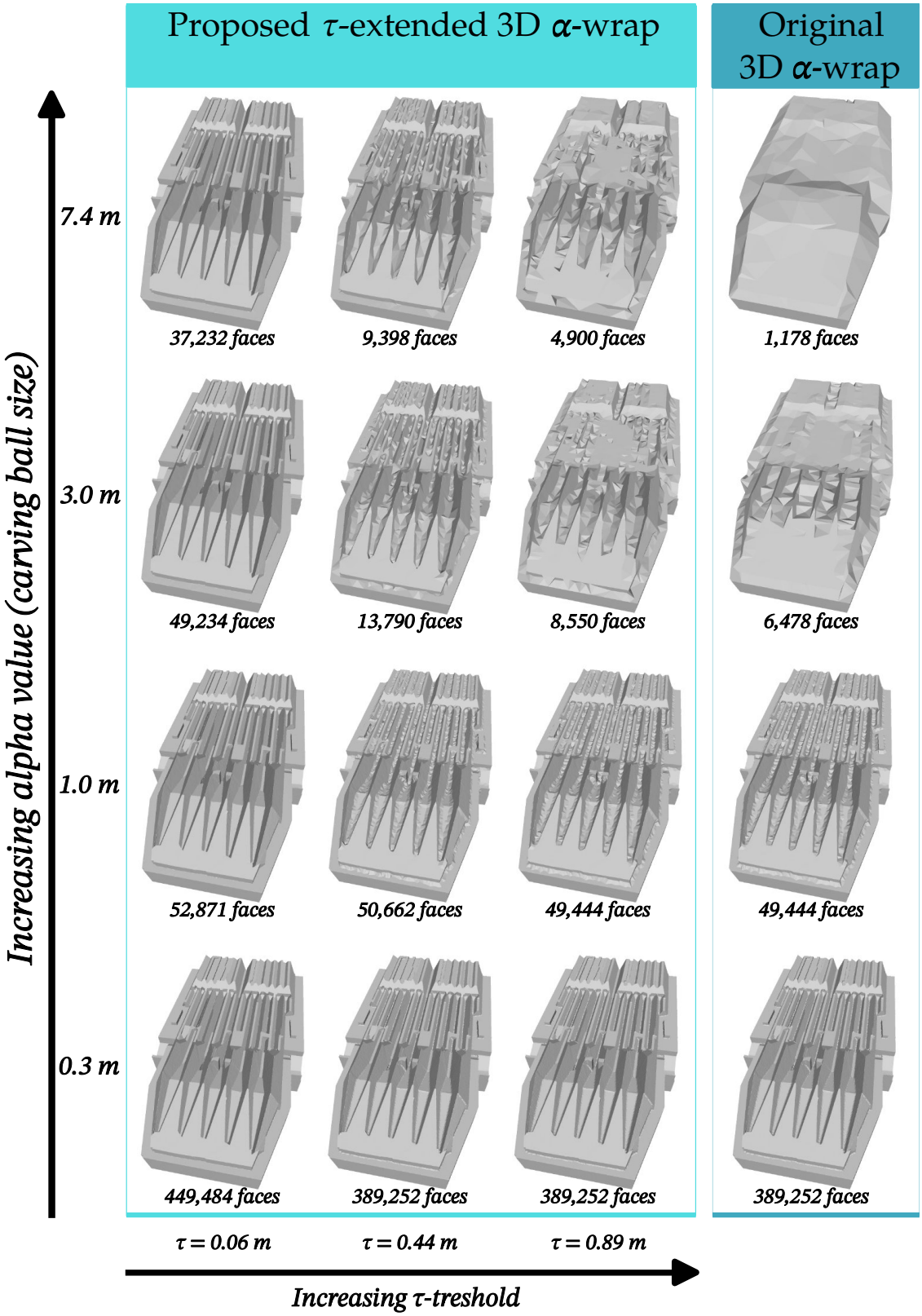


Figure E.1: Visual comparison of wrapped outputs for varying α- and τ-values on the Delft University of Technology Aula building from 3D BAG. The offset is fixed to 0.03m. Both methods are shown for α-values ranging from 0.3m to 7.4m. For the τ-extended method, τ is additionally varied from 0.06m to 0.89m. The figure shows the global effect of decreasing α and the additional local refinement introduced by the τ-criterion.

F Additional wrapping examples

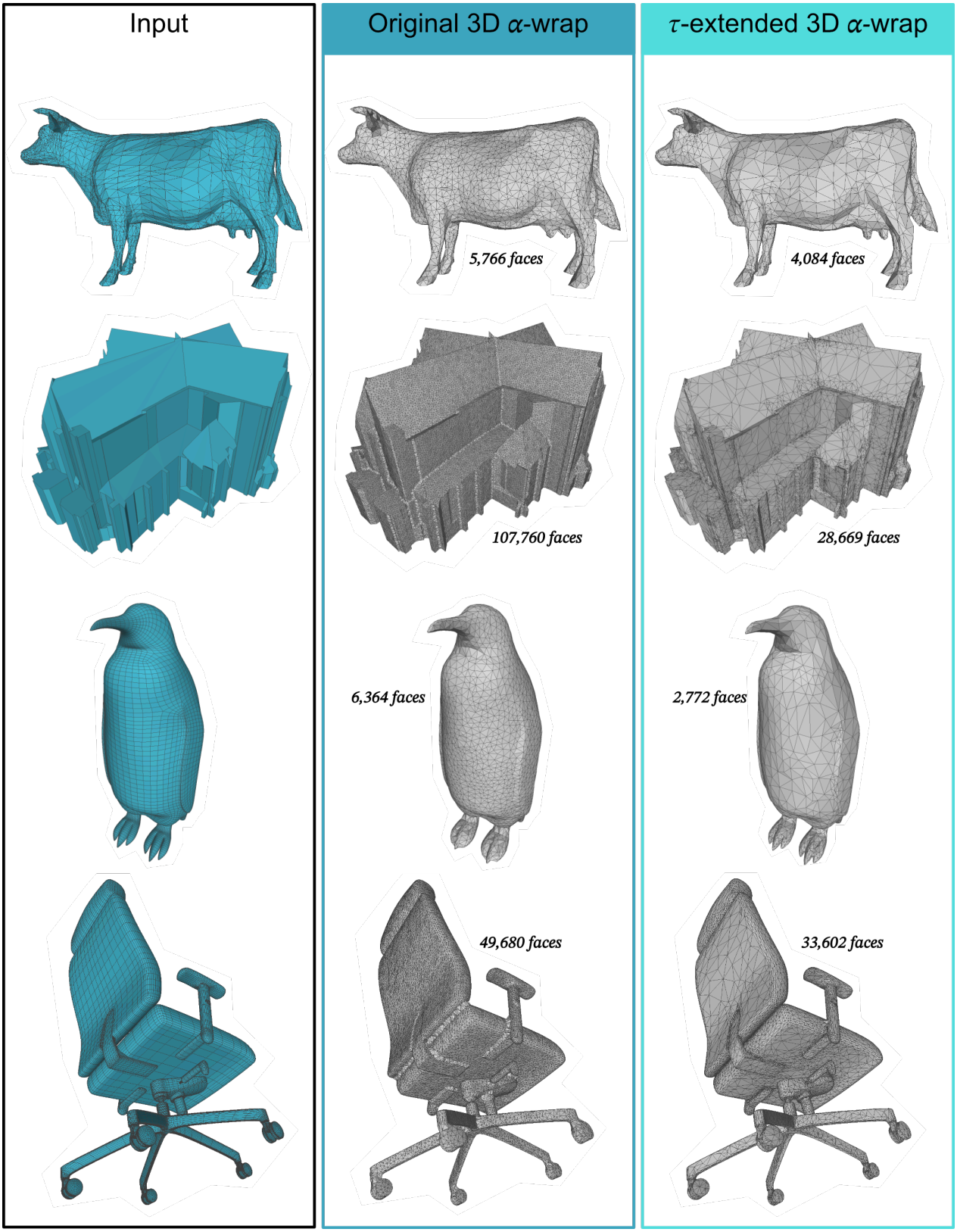


Figure F.1: Additional qualitative examples of wrapped outputs generated with the original 3D Alpha Wrap algorithm and the proposed τ-extended method. For each input geometry, the offset value is kept the same for both methods. The examples illustrate the type of meshes produced by both methods on different input geometries.